Plataforma de créditos y servicios

Documentación técnica para revisión

Estado: versión inicial, sin ejecutar contra base real. Pendiente de pruebas e integración de pagos.

1. Resumen

Sistema personal y privado para vender planes mensuales de créditos a clientes de marcas de indumentaria (fotografía editorial y producción audiovisual). El cliente no se registra: el administrador crea la cuenta, registra el pago y activa el saldo. El sistema gestiona el libro de créditos, los plazos para elegir servicios, la agenda y las decisiones manuales de baja, reactivación y eliminación, con historial auditable.

La regla de diseño central: **el servidor es la única autoridad**. El navegador nunca escribe datos de negocio, y toda mutación pasa por una función SQL transaccional, idempotente y con bloqueo por cliente.

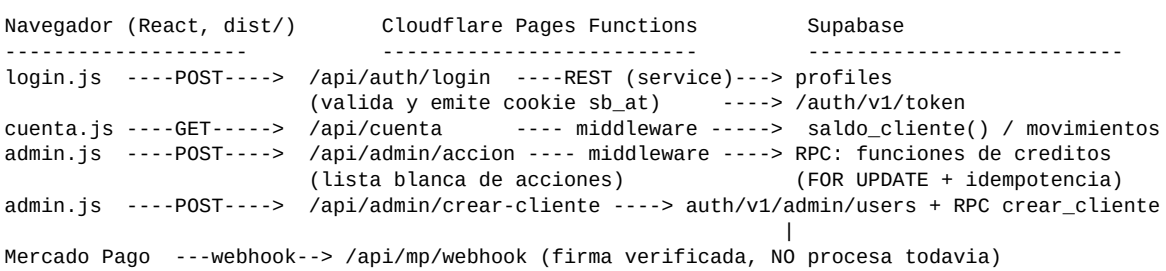
2. Stack tecnológico

Capa	Tecnología	Uso
Hosting y edge	Cloudflare Pages + Pages Functions	Sitio compilado en *dist/* y API en *functions/*, ambos desplegados desde Git
Runtime backend	JavaScript ES modules sobre Workers runtime (V8)	Handlers *onRequestGet/Post*, middleware con *onRequest*, Web Crypto para HMAC
Base de datos	PostgreSQL 15+ (Supabase, región São Paulo)	Tablas, funciones PL/pgSQL, RLS, pg_cron opcional
Autenticación	Supabase Auth (email + contraseña)	Login por usuario que se traduce a correo; tokens JWT validados contra /auth/v1/user
Frontend	React 18 + Vite (multi-página)	Tres páginas compiladas a *dist/*: ingreso, cuenta y administración. Sin router: cada página tiene su punto de entrada
Pagos (preparado)	API de Mercado Pago (Checkout Pro)	Verificación de firma x-signature y consulta de pagos; sin activar
Herramientas	Vite 5, Wrangler 4, Node 18+	Build del frontend, desarrollo local y deploy; el deploy productivo es por Git

Decisión: React con Vite en lugar de HTML plano. El panel tiene estado compartido entre secciones (avisos, clientes, formularios que dependen de otros datos), y React evita manipular el DOM a mano. Se mantiene Vite en modo multi-página para no necesitar un router en el cliente.

3. Arquitectura

Tres capas con responsabilidades separadas. El navegador solo conoce la anon key; la service role key vive en variables de entorno cifradas de Cloudflare y solo la usan los handlers de *functions/*.



Flujo de autenticación. El login busca el correo por usuario en *profiles*, llama a */auth/v1/token* con la anon key y guarda el access token en una cookie *sb_at* con HttpOnly, Secure, SameSite=Strict y Max-Age igual a su expiración. En cada request, el middleware reenvía el token a */auth/v1/user*, carga el perfil con la service key y verifica el rol. Un perfil en estado *eliminada* se rechaza aunque el token sea válido.

Defensa en profundidad. Hay tres capas: el middleware de Pages (rol admin en la API), la lista blanca de acciones en *accion.js* (solo se invocan funciones declaradas) y las propias funciones SQL, que llaman a *_requiere_admin(p_actor)*. El actor siempre sale de la sesión, nunca del cuerpo de la petición.

4. Estructura del repositorio

```
creditos/
  src/                        código React
    login/                      pantalla de ingreso
    cuenta/                     panel del cliente
    admin/                      panel de administración (Admin, Clientes, Formularios)
    shared/                     cliente HTTP, formatos, mensajes de error
    styles.css
  index.html, cuenta/, admin/ puntos de entrada de Vite
  vite.config.js              build multi-página hacia dist/
  functions/                  Pages Functions: cada archivo = ruta
    api/auth/login.js, logout.js
    api/cuenta/_middleware.js, index.js
    api/admin/_middleware.js, resumen.js, accion.js, crear-cliente.js
    api/mp/webhook.js           preparado, inactivo
  lib/                        codigo compartido (fuera de functions/ para no ser rutas)
    supabase.js                 cliente REST/RPC con service role; ErrorNegocio
    session.js                  cookie, verificacion de sesion
    http.js                     json(), manejar() para errores
    mercadopago.js              firma, consulta y preferencias (inactivo)
  supabase/
    migrations/0001..0004 .sql  esquema, RLS, reglas de creditos, altas
    tests/creditos_test.sql     pruebas en transaccion con ROLLBACK
  DEPLOY.md, README.md, package.json, .env.example
```

5. Modelo de datos

Tablas principales. Todas tienen RLS habilitado y ninguna política de escritura para clientes.

Tabla	Responsabilidad	Restricciones relevantes
profiles	Perfil de cliente o admin, enlazado a auth.users	username unico y con regex; estado en 4 valores; plazo_horas in (24,48)
plans / services	Catalogo editable desde el panel	precio > 0, creditos > 0, costo > 0; soft delete con *activo*
monthly_plans	Plan contratado por cliente y mes	unique(profile_id, periodo): un plan por mes; precio copiado al comprar
payments	Pagos pendientes, aprobados o rechazados	metodo en (mercado_pago, transferencia); mp_payment_id unico
credit_lots	Lotes de credito con vencimiento propio	restante <= inicial; gracia_hasta = ciclo_fin + 72 h
credit_movements	Libro de solo escritura	cantidad con signo; motivo >= 3 caracteres; idempotency_key unica
service_orders / order_items	Pedido de servicios con snapshot de precio y nombre	total > 0; items copian costo al momento de la compra
schedule_events	Agenda: rodaje, postproduccion, entrega	tipo en lista cerrada; ligado a un pedido
admin_audit	Auditoria de acciones administrativas	actor, accion, motivo, detalle jsonb
admin_notifications	Avisos para el panel	clave unica: evita avisos duplicados
operaciones_idempotentes	Resultado de cada operacion ya ejecutada	clave primaria; sin acceso por RLS

Tabla	Responsabilidad	Restricciones relevantes
terms_versions / acceptances	Versiones de terminos y aceptacion	existe el modelo; el flujo de aceptacion no esta implementado

Saldo como vista derivada. El saldo utilizable es la suma de *cantidad_restante* de los lotes *vigentes* con *gracia_hasta* > *now()*, calculada en *saldo_cliente()*. No hay un contador separado que pueda desincronizarse. El libro de movimientos permite reconstruir el estado completo.

6. Reglas de creditos (0003_creditos.sql)

Todas las operaciones siguen el mismo patron: verificar rol, verificar motivo si corresponde, consultar la clave de idempotencia, bloquear la fila del cliente con *FOR UPDATE*, validar estado y modificar lotes, movimientos, estado y auditoria, y guardar el resultado en *operaciones_idempotentes*. Un error revierte la transaccion completa.

Funcion	Quien	Efecto
crear_plan_mensual	admin	Crea plan pendiente de pago. Falla si ya existe uno en el mes.
registrar_pago	admin	Registra pago pendiente. Genera aviso.
aprobar_pago_y_activar	admin	Aprueba pago, crea lote, acredita, activa cuenta y arranca plazo de 48 h.
agregar_creditos	admin	Lote de ampliacion dentro del ciclo activo, con motivo.
confirmar_pedido	servidor (cliente autenticado)	Valida plazo y saldo; descuenta FIFO por vencimiento; crea pedido e items.
dar_de_baja	admin	Pierde todo el saldo con movimiento por lote y cierra el plan.
reactivar	admin	Plazo de 24 h y devolucion parcial acotada a lo efectivamente perdido.
eliminar_cuenta	admin	Pierde saldo, bloquea estado eliminada y conserva el historial.
vencer_creditos	cron o manual	Expira lotes cuya gracia termino. Idempotente por lote.
generar_avisos	cron o al abrir el panel	Crea avisos de plazo vencido, plazo por vencer y saldo por vencer. No cambia estados.

Concurrencia. El bloqueo por fila del perfil serializa las operaciones del mismo cliente. El descuento FIFO bloquea los lotes con *FOR UPDATE* ordenados por *gracia_hasta* y *created_at*, y consume hasta completar el total. Si el saldo no alcanza dentro del bucle, la transaccion se revierte.

Idempotencia. Cada operacion recibe una clave con prefijo por tipo (*pedido:*, *activar:*). Repetirla devuelve el resultado original. Los movimientos tienen claves propias (*acred:*, *consumo:pedido:lote*, *vence:lote*) que la base hace unicas.

Decisiones manuales. Ninguna funcion cambia el estado de un cliente por su cuenta salvo los vencimientos de saldo, que son un calculo sobre lotes y no una decision sobre la cuenta. Los plazos se calculan y generan avisos; el administrador decide.

7. Seguridad

- **RLS en todas las tablas.** Las politicas de lectura filtran por *profile_id = auth.uid()* o por *es_admin()*. No hay politicas de insert, update ni delete para roles de cliente.
- **Permisos de funciones.** Se revocan a public, anon y authenticated. Solo *service_role* puede ejecutarlas; *es_admin* queda accesible para que las politicas funcionen.
- **Service role solo en servidor.** Se usa en *lib/supabase.js*. El frontend solo recibe la anon key, que no concede escrituras.

- **Cookie de sesion.** HttpOnly, Secure, SameSite=Strict. SameSite cubre CSRF para los endpoints que mutan estado, porque aceptan solo JSON y el navegador no la envia desde otros sitios.
- **Errores.** Las funciones lanzan codigos de negocio en mayusculas. *lib/supabase.js* solo expone esos codigos; cualquier otro error se registra en el log y responde ERROR_INTERNO.
- **Sin enumeracion de usuarios.** Login devuelve siempre el mismo error. Los perfiles eliminados no pueden iniciar sesion.
- **Render seguro.** El frontend usa textContent y createElement; no inserta HTML proveniente de la base.
- **Alta de cliente atomica en dos sistemas.** Primero Supabase Auth, luego el perfil. Si el perfil falla, se borra el usuario de Auth.

8. Flujo operativo

Paso	Actor	Accion en el sistema
1	Admin	Crea cliente: login en Auth + perfil + auditoria
2	Admin	Crea plan del mes; el cliente paga por transferencia o Mercado Pago
3	Admin	Registra el pago. Aparece en avisos y en pagos pendientes
4	Admin	Aprueba y activa: acredita creditos, arranca plazo de 48 h
5	Cliente	Ingresa a /cuenta, elige servicios; se descuenta el saldo y se crea el pedido
6	Admin	Coordina fechas por fuera del sistema y carga la agenda (schedule_events)
7a	Sistema	Si vence el plazo, genera aviso. El admin decide: ampliar plazo, dar de baja o no hacer nada
7b	Admin	Baja: pierde saldo con motivo. Reactivar: plazo de 24 h y devolucion opcional
7c	Admin	Eliminar: bloquea el acceso y conserva el historial

9. Despliegue

Cloudflare Pages conectado a un repositorio privado de GitHub. Configuracion: sin framework, comando de build *npm run build*, directorio de salida *dist*. Cada push a la rama principal despliega. Las variables *SUPABASE_SERVICE_ROLE_KEY*, *MP_ACCESS_TOKEN* y *MP_WEBHOOK_SECRET* se cargan como secretas. Las migraciones **no** se aplican automaticamente: se ejecutan en el SQL Editor de Supabase en orden 0001 a 0004. Local: *npm run dev* con *.dev.vars*. El detalle operativo esta en DEPLOY.md.

10. Pruebas

supabase/tests/creditos_test.sql ejecuta once escenarios dentro de una transaccion que termina con ROLLBACK: activacion, idempotencia, ampliacion y consumo FIFO, saldo insuficiente, baja, validacion de motivo, devolucion excesiva, plazo vencido, avisos sin duplicar, vencimiento sin duplicar y permisos.

Estado: no ejecutadas. Escritas pero no corridas contra una instancia real de PostgreSQL. Es el primer control que debe hacerse antes de cualquier otra revision.

11. Riesgos conocidos y deuda tecnica

Lista honesta de lo que revisaria yo antes de usar datos reales. Ordenada por prioridad.

Prioridad	Tema	Detalle
Alta	Idempotencia fuera del bloqueo	En *confirmar_pedido* y en varias funciones, la consulta de *operaciones_idempotentes* ocurre antes de tomar el bloqueo del perfil. Dos llamadas simultaneas con la misma clave podrian pasar ambas la verificacion y descontar dos veces. Correccion: mover la consulta despues del *FOR UPDATE*. Pendiente de aplicar.
Alta	Pruebas no ejecutadas	Ninguna prueba corrio todavia. Hasta correrlas, la logica de creditos es un diseno, no un hecho verificado.
Alta	Respaldos	No hay copia propia ni plan de restauracion. Recomendado: plan Pro de Supabase y pg_dump diario a R2.
Alta	Login en producción sin validar	El sitio está publicado en Cloudflare Pages, pero el login de administrador no pudo verificarse todavía (credenciales rechazadas). Diagnóstico pendiente: revisar el proveedor de correo de Supabase, la clave anon en Cloudflare y los logs de funciones.
Media	Sesion sin refresh	La cookie dura lo que dura el access token (aprox. 1 h). Tras expirar, hay que volver a ingresar.
Media	Sin rate limiting	El login no tiene limite de intentos propio. Mitigacion: reglas de Cloudflare sobre /api/auth/login.
Media	Traslado de 1.000 creditos	Especificado y no implementado. Requiere funcion de cierre de ciclo y bandera de no trasladable.
Media	Aceptacion de terminos	Tablas creadas, flujo sin implementar. Debe ser bloqueante antes del primer pago.
Media	Bloqueo en Auth al eliminar	La eliminacion bloquea en la base y el login, pero no revoca tokens activos en Auth. Mitigacion: ban del usuario con la API admin.
Baja	Lotes vencidos al dar de baja	Lotes vigentes con gracia ya vencida se registran como perdida y no como vencimiento. Contablemente ambos son saldo perdido; conviene decidir cual usar.
Baja	Alta con error parcial	Si falla el borrado compensatorio en Auth, queda un usuario huerfano. Registrar y reintentar.
Baja	Mercado Pago	Preparado, no conectado. Verificar el formato de firma contra la documentacion vigente y probar con credenciales de prueba.

12. Preguntas para revision

- Es correcto usar funciones SQL con *security definer* solo en *es_admin*, o conviene que todas las mutaciones sean invoker con permisos finos?
- La idempotencia por tabla de claves: ¿preferis moverla a una columna con restriccion unica por operacion en vez de tabla separada?
- ¿Conviene pg_cron dentro de Supabase o un Worker con Cron Trigger en Cloudflare para los procesos de vencimiento?
- React con Vite: ¿preferirías Next.js o Astro (render en servidor) para las páginas públicas futuras?
- Para la firma de Mercado Pago, ¿validarias tambien la antiguedad del timestamp (ventana de replay)?

Anexo: reglas de negocio resumidas

- Un plan por cliente y por mes calendario de compra.
- Ciclo de 30 dias desde la activacion. Saldo vencido tras 72 h de gracia al terminar el ciclo.
- Plazo para elegir servicios: 48 h tras activar; 24 h tras reactivar. Vencido el plazo, el sistema avisa y el admin decide.
- Descuento FIFO por fecha de vencimiento mas cercana. Los creditos no son dinero, no se transfieren ni se reembolsan.
- Baja y eliminacion con motivo obligatorio. El historial se conserva hasta que el admin decida borrarlo.
- Ampliacion de creditos dentro del ciclo activo, como movimiento separado del plan.